

Welcome to Today's R Lab!

Today's Agenda

1. **Course Outline**
2. **R Basics** — What is R?
3. **Vectors & Arrays** — R's primitive types
4. **Operators** — Operations used with vectors
5. **Control Flow** — if, else, else if, switch, for
6. **Functions** — R function syntax
7. **Matrices & Arrays** — Multi-dimensional vectors
8. **Sampling**
9. **Sampling Exercises**

What is R?

R is an open-source interpreted functional programming language (with OO elements) used for statistics and data analysis.

Vectors & Arrays

There are 6 atomic data types in R:

Type	Description	Example
Double	Double-precision floating-point numbers	3.14, 42.0
Integer	Whole numbers (denoted explicitly with an L suffix)	42L, -7L
Character	Strings	"hello", 'data'

Type	Description	Example
Logical	Boolean values	TRUE, FALSE (or T, F)
Complex	Numbers with real and imaginary parts	2 + 3i
Raw	Raw bytes / hexadecimal values	as.raw(42)

Note: “Numeric” is a superclass that combines both **Double** and **Integer**.

Note: All scalars in R are simply 1-dimensional vectors.

Atomic Vectors

Can only contain elements of the same data type.

Created using the combine function `c()`.

```
x <- c(10, 20, 30, 40)
x
# [1] 10 20 30 40
```

The combine function automatically flattens nested vectors:

```
x <- c(c(1, 2), c(3, 4))
x
# [1] 1 2 3 4
```

Coercion

Because atomic vectors can only contain elements of a single data type, R automatically converts (coerces) elements if you attempt to combine different types together.

The coercion hierarchy

Logical -> Integer -> Double (Numeric) -> Complex -> Character

Examples

Logical and Integer -> Integer (TRUE becomes 1)

```
c(TRUE, 5L)  
#> [1] 1 5
```

Integer and Numeric -> Double (Numeric)

```
c(1L, 2.5)  
#> [1] 1.0 2.5
```

Numeric and Character -> Character

```
c(1, 2.5, "apple")  
#> [1] "1"      "2.5"    "apple"
```

Assignment operators

Operator	Meaning
<-	Assign right to left
=	Assign right to left
<<-	Global (parent environment) assignment operator
->	Left-to-right version of <-
->>	Left-to-right version of <<-

Example

```
mu <- 5  
sigma = 2  
10 -> n
```

The difference between `<-` and `=` shows up inside function calls:

```
mean(x = c(1, 2, 3))    # x is an argument name; no variable  
x                        # Error: object 'x' not found  
  
mean(x <- c(1, 2, 3))    # assigns x in your workspace AND pa  
x  
# [1] 1 2 3
```

Operators

Most operators in R work on vectors element-wise.

Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
^	Exponent	5 ^ 2	25
%%	Modulo (remainder)	5 %% 2	1
%/%	Integer division	5 %/% 2	2

When vectors have different lengths, the shorter one is **recycled**:

```
c(1, 2, 3, 4) * c(10, 100)
#> [1] 10 200 30 400
```

R warns you if the longer length isn't a multiple of the shorter one:

```
c(1, 2, 3) + c(10, 20)
# [1] 11 22 13
# Warning message:
# In c(1, 2, 3) + c(10, 20) :
# longer object length is not a multiple of shorter object
```

Comparison

Comparison operators return logical vectors: ==, !=, <, >, <=, >=.

```
x <- c(5, 10, 15, NA)
x > 8
#> [1] FALSE TRUE TRUE NA
x == 10
#> [1] FALSE TRUE FALSE NA
```

Most comparison involving NA returns NA, because R can't know the answer.

Logical

Operator	Meaning	Works on
!	NOT	vectors
&	AND	vectors (element-wise)
	OR	vectors (element-wise)
xor()	Exclusive OR	vectors
&&	AND	single values only
	OR	single values only

Examples

```
age <- c(22, 35, 41, 58)
age > 30 & age < 50
#> [1] FALSE TRUE TRUE FALSE
age < 25 | age > 55
#> [1] TRUE FALSE FALSE TRUE
!(age > 30)
#> [1] TRUE FALSE FALSE FALSE
```

Comments:

Use `&&` and `||` in if conditions, where you need a single TRUE or FALSE. Use `&` and `|` when filtering vectors.

Logical values are treated as 0 and 1 in arithmetic, which gives you counts and proportions:

```
sum(age > 30)      # how many  
#> [1] 3
```

```
mean(age > 30)     # proportion  
#> [1] 0.75
```

Miscellaneous operators

Membership: %in%

Tests whether each element on the left appears in the vector on the right.

```
c("a", "z") %in% c("a", "b", "c")  
#> [1] TRUE FALSE
```

Sequence operator :

```
1:5  
#> [1] 1 2 3 4 5  
5:1  
#> [1] 5 4 3 2 1
```

Indexing and extraction

Operator	Use	Example
[]	Subset a vector, matrix, or data frame	<code>x[2], x[-1], x[x > 2]</code>
[[]]	Extract a single element from a list	<code>list[["a"]]</code>
\$	Extract a named element or column	<code>df\$height</code>

Note: $x[i]$ and $x[[i]]$ are equivalent when used on vectors and arrays.

Note: Indexing starts from 1.

Note: $x[]$ returns the entire vector.

Examples

```
x <- c(a = 10, b = 20, c = 30)
```

```
x[2]                # by position
```

```
#>  b
```

```
#> 20
```

```
x[-1]              # drop the first
```

```
#>  b  c
```

```
#> 20 30
```

```
x[c(1, 3)]         # list of integers
```

```
#>  a  c
```

```
#> 10 30
```

Examples

```
x[c(T, T, F)]    # list of logicals
```

```
#>  a  b
```

```
#> 10 20
```

```
x[x > 15]        # by condition
```

```
#>  b  c
```

```
#> 20 30
```

```
x["c"]           # by name (if named)
```

```
#>  c
```

```
#> 30
```

Control flow

If-else

```
if (x > 90) {  
  "A"  
} else if (x > 80) {  
  "B"  
} else if (x > 50) {  
  "C"  
} else {  
  "F"  
}
```

Since if returns a value, you can also use it as a ternary operator:

```
x1 <- if (TRUE) 1 else 2  
x2 <- if (FALSE) 1 else 2  
  
c(x1, x2)  
#> [1] 1 2
```

ifelse() - a vectorized version of if

```
x <- 1:10  
ifelse(x %% 5 == 0, "XXX", as.character(x))  
#> [1] "1" "2" "3" "4" "XXX" "6" "7" "8" "9"
```

Switch-case

```
switch(x,  
  a = "option 1",  
  b = "option 2",  
  c = "option 3",  
  "option 4" # default value  
)
```

Note: *Since the focus of the labs is statistic we'll be avoiding explicit looping in favour of functionals.*

For-loops

```
for (i in 1:3) {  
  print(i)  
}  
#> [1] 1  
#> [1] 2  
#> [1] 3
```

The *next* and *break* keywords function like in C++

Related tools:

`while(condition) action`: performs action while condition is TRUE.

`repeat(action)`: repeats action forever (or until it encounters `break`).

Functions

Functions are first-class objects. You create them with `function()` and assign them to a name, or you can use them as anonymous functions.

```
square <- function(x) {  
  x^2  
}
```

```
square(4)  
#> [1] 16
```

```
(function(x) x + 1)(2)  
#> [1] 3
```

The value of the last evaluated expression is returned automatically. You can return explicitly with `return()`, e.g. if you need the function to exit early.

Arguments

Arguments can have **default values** and can be passed by **name** or by **position**.

```
power <- function(x, exponent = 2) {  
  x^exponent  
}
```

```
power(3)                                # default exponent
```

```
#> [1] 9
```

```
power(3, 3)                             # by position
```

```
#> [1] 27
```

```
power(exponent = 3, x = 2)              # by name, any order
```

```
#> [1] 8
```

Functions are vectorized automatically when they are built from vectorized operators:

```
power(1:5)
```

```
#> [1] 1 4 9 16 25
```

Matrices & Arrays

A **matrix** is an atomic vector with a `dim` attribute (rows and columns). It can only hold one data type, just like a vector.

```
m <- matrix(1:6, nrow = 2, ncol = 3)
```

```
m
```

```
#>      [,1] [,2] [,3]
```

```
#> [1,]    1    3    5
```

```
#> [2,]    2    4    6
```

R fills matrices **column by column**. Use `byrow = TRUE` to fill by row.

```
matrix(1:6, nrow = 2, byrow = TRUE)
```

```
#>      [,1] [,2] [,3]
```

```
#> [1,]    1    2    3
```

```
#> [2,]    4    5    6
```

Combine vectors or matrices with `rbind()` (stack rows) and `cbind()` (join columns):

```
rbind(c(1, 2, 3), c(4, 5, 6))
```

```
#>      [,1] [,2] [,3]
```

```
#> [1,]    1    2    3
```

```
#> [2,]    4    5    6
```

```
cbind(a = 1:3, b = c(10, 20, 30))
```

```
#>      a  b
```

```
#> [1,] 1 10
```

```
#> [2,] 2 20
```

```
#> [3,] 3 30
```

Indexing matrices

Use `matrix[row, column]`.

Note *Leaving one empty selects everything in that dimension.*

```
m[2, 3]      # single element
#> [1] 6

m[1, ]       # first row
#> [1] 1 3 5

m[, 2]       # second column
#> [1] 3 4

m[, c(1, 3)] # columns 1 and 3
#>      [,1] [,2]
#> [1,]    1    5
#> [2,]    2    6
```

Arrays

An **array** generalizes matrices to any number of dimensions.

```
a <- array(1:24, dim = c(2, 3, 4))  
dim(a)  
#> [1] 2 3 4
```

Note: A matrix is simply a 2-dimensional array, and a vector has no *dim* attribute at all.

Sampling

`sample(x, size, replace = FALSE, prob = NULL)` draws a random sample of elements from a vector.

Argument	Meaning
<code>x</code>	The vector to sample from
<code>size</code>	How many elements to draw (defaults to <code>length(x)</code>)
<code>replace</code>	Sample with replacement? (defaults to <code>FALSE</code>)
<code>prob</code>	Optional vector of sampling probabilities

Repeating a simulation: replicate()

replicate(n, expr) evaluates expr **n separate times** and collects the results. It is the standard tool for simulations. We can estimate a probability by taking the mean of a logical vector:

```
## Exercise 1
is_same_ball <- function() {
  box <- 1:8
  size <- 2
  replace <- T

  result <- sample(box, size, replace)
  result[1] == result[2]
}

number_reps <- 10 ^ 4
mean(replicate(number_reps, is_same_ball()))
```

Some more useful functions

`rep()` repeats the elements of a vector.

```
rep(c("a", "b"), times = 3)
#> [1] "a" "b" "a" "b" "a" "b"
```

`seq()` builds vectors

```
seq(from = 0, to = 1, by = 0.25)
#> [1] 0.00 0.25 0.50 0.75 1.00
```

`distinct()` shows which values are distinct

```
distinct(c(1, 2, 1))
#> [1] FALSE TRUE FALSE
```

`unique()` creates a vector of unique values from the given vector

```
unique(c(1, 2, 1))
#> [1] 1 2
```